

Games & AI Techniques

Week 2: Introduction to Steering Behaviors

Dr. Ginel Dorleon

Week 1 Recap



About this course

This course is about:

- AI Techniques applied to games
- We will cover more AI techniques than talking about games (80/20)
- We will explain you different AI Techniques and how they can be applied in some old school games
- Mastering the AI Techniques is the most important aspect of the course.

What this course is not about ?

- High level of Gaming Dev **X**
- This course is not about gaming **X**

What do I have to do in this course ?

Assignment:

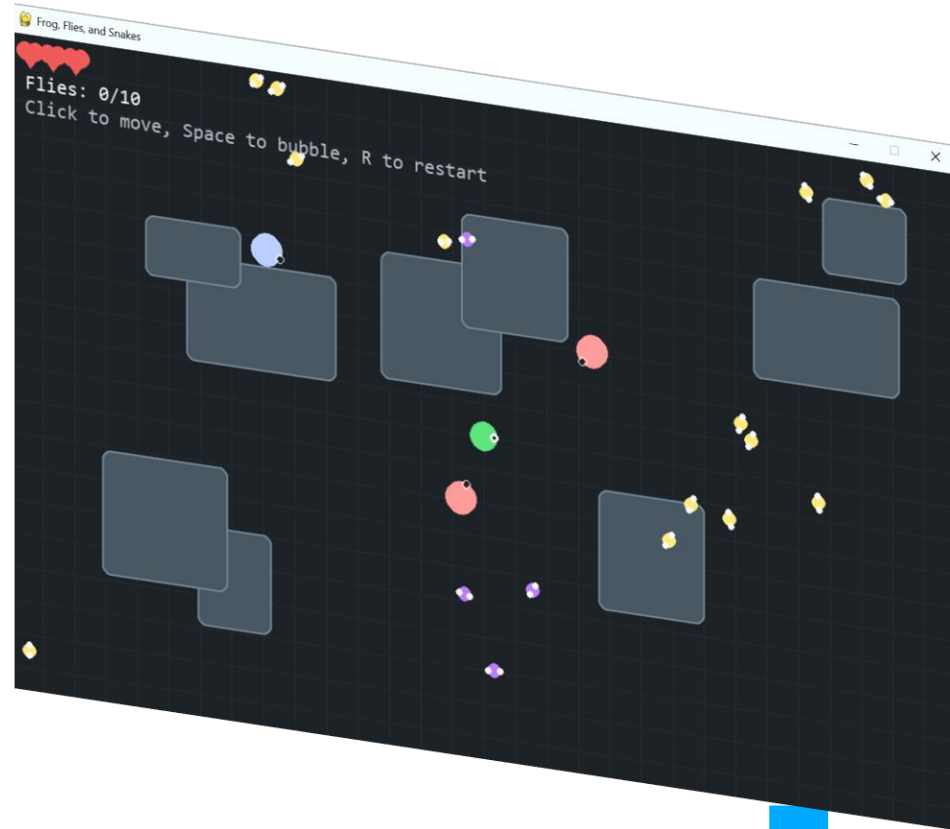
- You must complete 3 Assignments: 2 Individually, 1 in a group of 3-4
- Each assignment has a Technical Interview (we want to make sure you understand the techniques)
- 3 assignments => 5+ AI Techniques
- **A1: Steering Behaviors & FSM**, **A2: Path Planning & MCTS**, **A3: Reinforcement Learning**

Participation:

- Attend online lectures and the face-to-face tutorial. Ask question !!!
- Join discussion and build on others. Read extra source each week.
- **Self learning:** read beyond class, review after class. Apply learned concepts to assignments.

Assignment 1 is available on Canvas

- Predator/prey game with frog, flies, snakes.
- Focus is on old-school games + AI: **Steering behaviours + finite state machines.**
- A little bit game dev, since this is *games* and AI after all.
- Graphics, animations, and **some skeleton code is provided**. You need to implement the missing bits.
- Due at the end of week 4.
- Expectation: Do one of the 3 parts each week (starting today).



Today Agenda

- Steering Behaviors?
- The Physics Behind Steering
- Types of Steering Behaviors



What is Steering?

Let's imagine a small creature in a game: like a frog, a bird, or a car.

We want it to **move in a realistic and intelligent way**:

- The frog should move toward the fly.
- The bird should turn smoothly, not instantly.
- The car should avoid hitting obstacles and follow the road.

All these actions are examples of steering: → steering means deciding how to change direction and speed smoothly over time (steer: guide, direct, pilot, lead..)

Steering behavior is a rule that tells a game character how to move in a way that looks natural and intelligent, based on what it sees or wants to do.

What is Steering?

Real-life Example:

Think of driving a car or riding a bike:

- You don't instantly face a new direction: you turn the steering wheel gradually.
- You don't stop instantly: you press the brakes and slow down.
- You don't drive straight into a wall: you steer away.

So Games use the same idea:

- Characters don't jump from one place to another.
- They move, turn, and slow down using small steering corrections every frame.



Why We Use Steering in Games

Steering is used whenever we want believable motion:

Situation	Example in games	Steering type used
Move toward a goal	Frog moves to a clicked point	Seek
Move away from danger	Fly runs from snake	Flee
Stop smoothly at target	Frog slows near fly	Arrival
Chase a moving target	Snake hunts frog	Pursuit
Wander naturally	Flies buzz randomly	Wander
Avoid bumping into others	Snakes don't collide	Separation / Avoidance
Group movement	Birds flock together	Flocking



The Physics Behind Steering



The Tiny Physics Behind Steering

Understand how movement works in a game world.

Everything that moves has three Things:

- Every moving game object: a frog, a car, a spaceship, can be described by just 3 basic quantities:

	Symbol	Question it answers	Example
Position	p	"Where am I?"	(100, 200) on the screen
Velocity	v	"How am I moving?"	5 pixels per frame to the right
Acceleration	a	"How is my movement changing?"	speeding up, slowing down..

The Tiny Physics Behind Steering

How Movement Happens?

Imagine driving a car:

- **Position**: where your car is right now on the road.
- **Velocity**: how fast and in what direction we're driving/the speed of something given a direction!
- **Acceleration**: how your speed changes: pressing gas, braking, or turning the wheel.

Now in games, we update these things many times every second => frames per second or FPS.

- Acceleration changes velocity.
- Velocity changes position.

- **FPS**: how often the game updates each second.
- **Δ time** (delta time): the amount of time that passes between two frames(two updates).
- **Δt** is the inverse of **FPS** => **$\Delta t = 1 / \text{FPS}$**

The Tiny Physics Behind Steering



The Formulas:

For each frame, we do:

velocity = velocity + acceleration * dt

position = position + velocity * dt

Remember: dt means “delta time”, how many seconds have passed since the last frame (usually around 1/60 of a second).



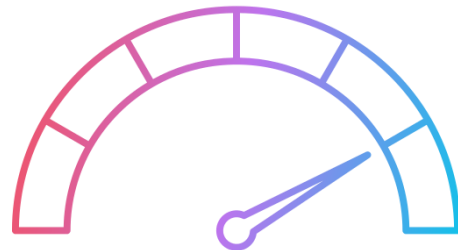
The Tiny Physics Behind Steering

The Role of Force (What Causes Acceleration)

Now we ask: what makes acceleration happen?

In real life, forces change our velocity:

- Gas pedal pushes the car forward.
- Brakes apply force backward.
- Turning the wheel changes the direction of the force.



In our game, steering behaviors are those forces!

- Each steering behavior computes a **steering force**: a small vector that says “turn this way” or “slow down a bit”.
- Then we convert that force into acceleration:
$$\text{acceleration} = \text{steering_force} / \text{mass}$$
if we treat mass = 1 (to keep things simple), then:
$$\text{acceleration} = \text{steering_force}$$



Understand Velocity and Direction

velocity is not just speed: it has direction too.

Example:

- A car moving right at 10 units per second: $velocity = (10, 0)$
- A car moving up at the same speed: $velocity = (0, -10)$ => *(y-axis increases downwards in Pygame)*

Limiting Speed and Force

To make motion realistic, we usually limit how strong or fast things can go.

- **max_speed** => how fast the agent can move
- **max_force** => how sharp it can turn

We use these limits so the agent can't instantly spin or move at light speed.

```
if |velocity| > max_speed:
    velocity = normalize(velocity) * max_speed
if |steering_force| > max_force:
    steering_force = normalize(steering_force) * max_force
```



The Physics Behind Steering

-

Resume



Resume

Velocity:

- Describes how fast and in what direction something is moving.
- If you know position over time: $velocity = (new_position - old_position) / \Delta t$
- In game loops (with known velocity), we update position: $position = position + velocity * \Delta t$

Acceleration:

- Describes how velocity is changing
- $acceleration = (new_velocity - old_velocity) / \Delta t$
- Used in motion updates: $velocity = velocity + acceleration * \Delta t$

Resume

Steering Force:

- The force needed to change current velocity toward the desired direction.
- $steering = desired_velocity - current_velocity$

Acceleration from Steering Force:

- Use Newton's Second Law (simplified for 2D games):
- $acceleration = steering_force / mass$
- If we assume mass = 1 (common in simple games): $acceleration = steering_force$

Learning Activity I

Learning Activity I - Motion & Steering in Games

Basic understanding of the concepts of position, velocity, acceleration

1- Acceleration changes an object's velocity over time.

- ☒ A) True
- ☐ B) False

2. Which of the following best describes a steering force in a game?

- ☐ A) A real-world force like gravity
- ☐ B) A velocity vector
- ☒ C) A computed force that changes how a character moves
- ☐ D) The direction the player is facing

Learning Activity I - Motion & Steering in Games

Basic understanding of the concepts of position, velocity, acceleration

3. An object with constant acceleration will keep speeding up or slowing down over time.

A) False

☒ B) True

4. What does this formula $steering = desired_velocity - current_velocity$ calculate?

A) Final position

B) Object's speed

C) Player input

☒ D) The needed change in movement

Learning Activity I - Motion & Steering in Games

Basic understanding of the concepts of position, velocity, acceleration

5-If acceleration is zero, velocity will not change.

- ☒ A) True
- ☐ B) False

6- What happens when a small steering force is applied every frame?

- ☒ A) The object instantly changes direction
- ☐ B) The object gradually curves toward a new direction
- ☐ C) The object stops moving
- ☐ D) The object teleports

Learning Activity I - Motion & Steering in Games

Basic understanding of the concepts of position, velocity, acceleration

7- Steering behaviors always produce the same result regardless of the object's current velocity.

A) True

☒ B) False

8- Which of the following will directly change an object's position?

A) Acceleration

☒ B) Velocity

C) Steering force

D) Mass

9- If no steering force is applied, and velocity is not zero, the object will keep moving in the same direction.

☒ A) True

B) False



Steering Behaviors Overview



Steering Behaviors Overview

- Steering Behaviors are designed to give autonomous characters the ability to navigate around their world in a life-like and improvisational manner.
- They are not based on complex strategies involving path planning or global calculations but instead use local information.



Seek Behavior



What is it:

- Seek makes the character move toward a target position: it tries to match its position with the target's position.

Goal/Use:

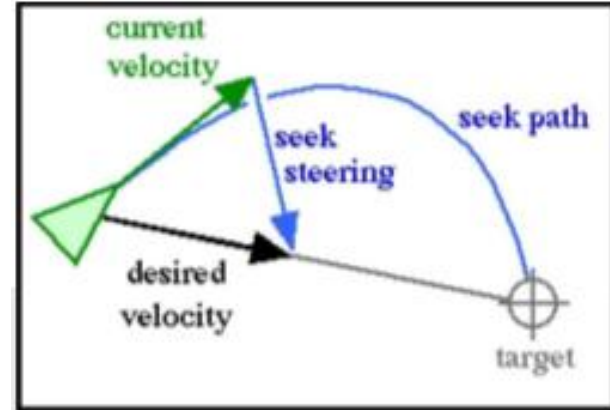
- Used when we want an agent/ to go to a specific point. For example, the frog moving toward a clicked location or a missile homing in on a target.

How It Works:

- Find the direction from the agent to the target.
- Turn that direction into a desired velocity : “this is how I want to move.”
- Compute the steering force = desired velocity - current velocity.
- Apply that steering force to slowly turn and move toward the target.

Seek Behavior

- The green triangle = the agent (character).
- The circle with a cross = the target.
- The green arrow = current velocity (where it's moving now).
- The black arrow = desired velocity (direction of the target).
- The blue arrow = seek steering (the correction to face the target).
- The curved path = how the agent turns smoothly to reach the target.



Flee Behavior

What is it:

- Flee is the opposite of Seek: it makes the character move away from a threat or target

Goal/Use:

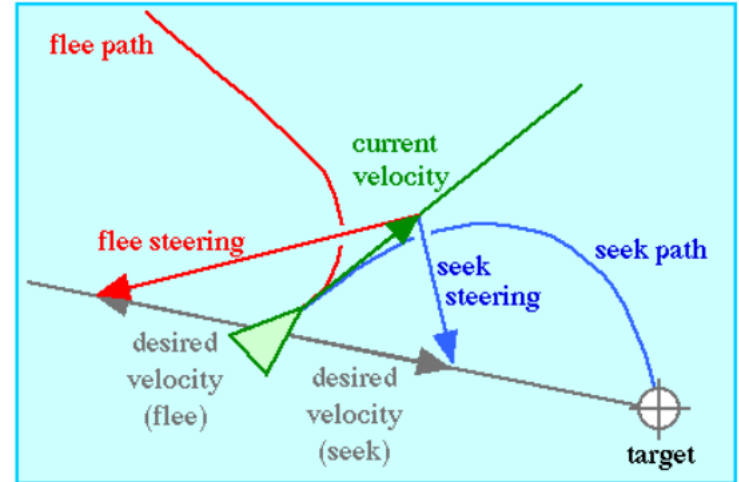
- Used when an agent must escape danger, like a fly running away from a snake.

- **How It Works:**

- Find the direction away from the target (reverse of Seek).
- Set that as the desired velocity.
- The steering force turns the agent away from the threat.

Flee Behavior

- Blue arrows show Seek forces and path (toward target).
- Red arrows show Flee forces and path (away from target).
- Both use the same logic: just the direction is reversed.



Arrival Behavior

What is it:

- Arrival is a gentler version of Seek. It makes the agent slow down as it gets close to the target.

Goal/Use:

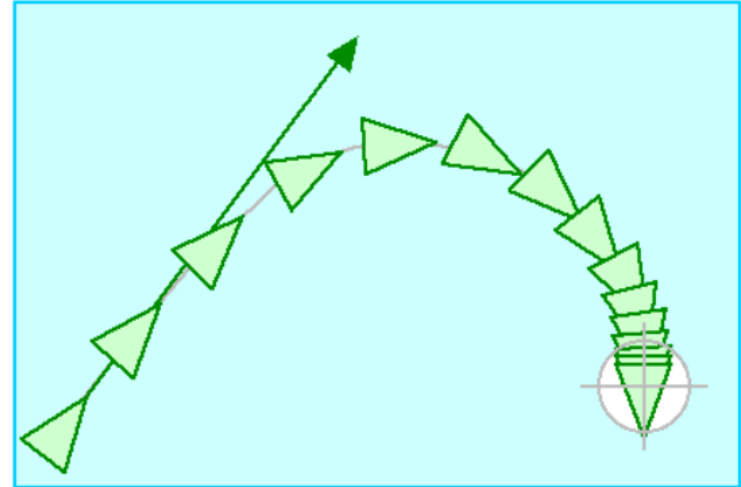
- Used when you want your character to stop smoothly, like a car braking before a stop sign or the frog slowing down near a fly.

- **How It Works:**

- Compute the desired velocity like Seek.
- If the target is far away, move at full speed.
- If it's close, reduce speed proportionally to distance.
- The closer you are, the slower you move: until you stop exactly at the target.

Arrival Behavior

- The triangles show the frog's position at different times.
- They get closer together near the target, showing it's slowing down.
- The green arrow shrinks near the target: smaller velocity = slower movement.
- The circle around the target is the slowing radius.



Pursue and Evade Behavior

What is it:

- **Pursue:** Chase a moving target by predicting where it will be, not just where it is.
- **Evade:** Escape from a moving pursuer by predicting where it will reach you and avoiding that point.

Goal/Use:

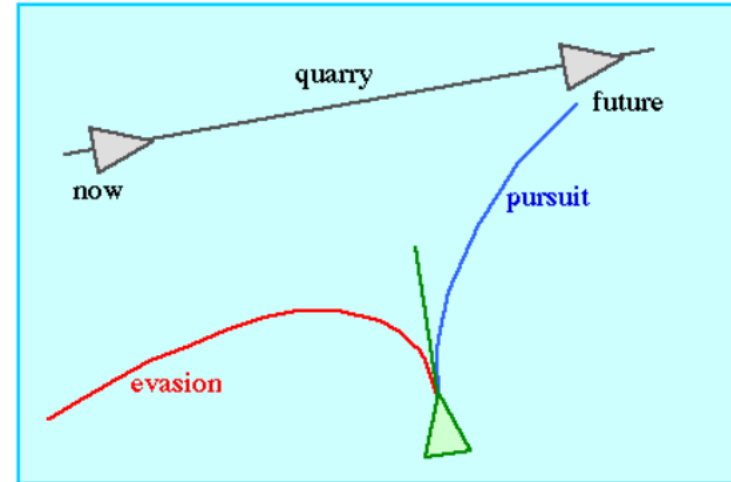
- Used when both the hunter and the prey are moving, like a snake chasing a frog or a frog dodging an incoming snake.

How It Works:

- Measure the target's velocity (how fast it's moving).
- Predict its future position after a short time.
- Seek (for Pursue) or Flee (for Evade) that future position.

Pursue and Evade Behavior

- The gray triangle (quarry) shows where the target is now and where it will be in the future.
- The blue curve (pursuit) shows the chaser's predicted path.
- The red curve (evasion) shows how the prey bends away from the predicted collision point.



Wander Behavior

What is it:

- Wander gives the agent a natural-looking random movement, it changes direction gradually, not instantly.

Goal/Use:

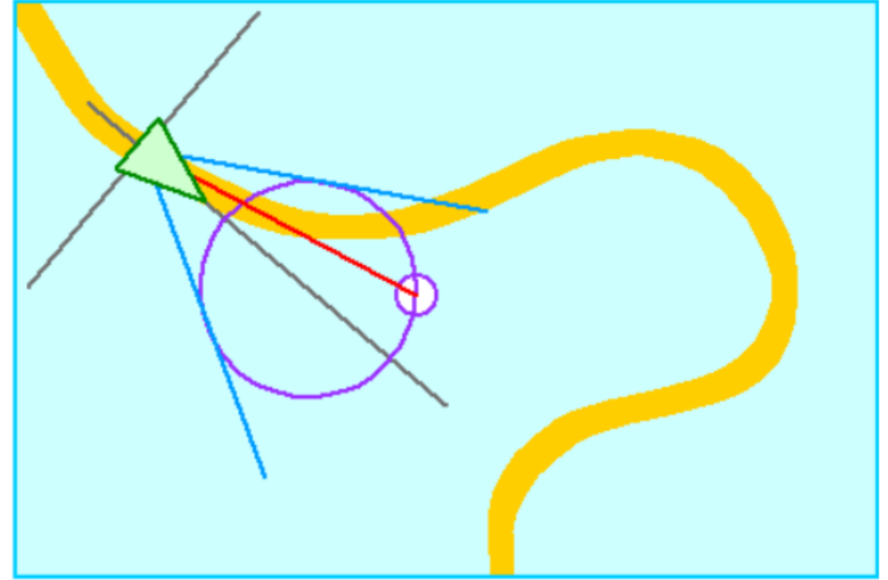
- Used to make background characters or animals move in a relaxed, natural, and unpredictable way.(Example: flies buzzing randomly.)

- **How It Works:**

- Place a circle ahead of the agent in the direction it's moving.
- Pick a random point on that circle as the new temporary target.
- Seek that point each frame.
- Move the point slightly each frame so the direction drifts smoothly.

Wander Behavior

- The purple circle = the “wander circle,” located a short distance ahead.
- The red line = the chosen random direction.
- The yellow line/path = the final curved wandering path.



Path Following

What is it:

- Path Following keeps the agent close to a predefined path or road.

Goal/Use:

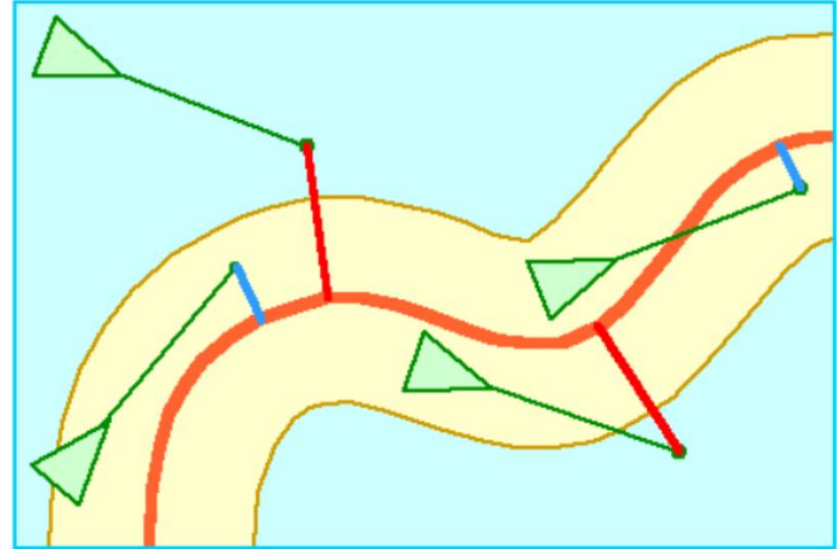
- Used for objects that must follow a trail, river, or patrol route without leaving it.

- **How It Works:**

- Predict where the agent will be soon (look ahead).
- Find the closest point on the path to that future position.
- Set a target point slightly ahead on the path.
- Seek that target to stay near the path center.

Path Following

- The orange line = path center.
- The light-yellow band = allowed area near the path.
- The red lines = perpendicular distances from agent to path (correction).
- The blue dots = points the agent should aim for.
- The green triangles = agent positions as it follows the curve.



Collision Avoidance

What is it:

- Avoid running into other moving agents by predicting future collisions.

Goal/Use:

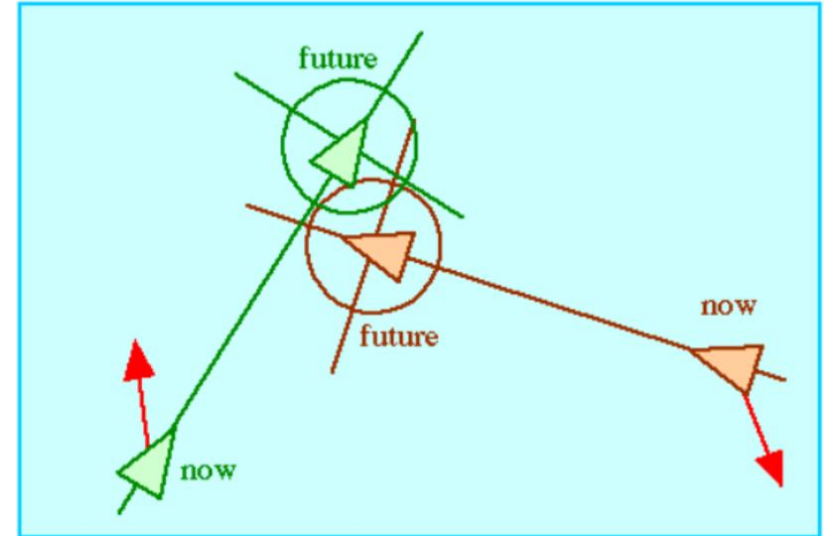
- Used in crowds, traffic, and swarms to prevent bumping or overlaps.

- **How It Works:**

- Look at other agents' positions and velocities.
- Predict where each will be in the next few moments.
- If a collision seems likely, apply a steering force sideways or backward.

Collision Avoidance

- The green and brown triangles = two moving agents.
- “Now” = current positions; “Future” = predicted positions.
- The red arrow = avoidance steering applied by the green agent.
- The goal: move so their future circles do not overlap.



Obstacle Avoidance

What is it:

- Avoid hitting fixed obstacles like walls, rocks, or trees.

Goal/Use:

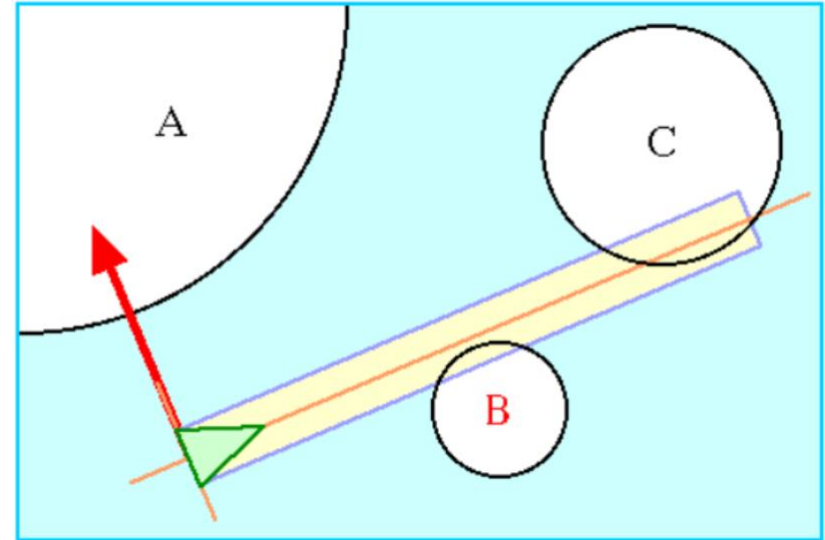
- Used for navigating spaces with static objects.

- **How It Works:**

- Cast feelers (short rays) ahead of the agent.
- If a feeler detects an obstacle, compute a steering force that pushes the agent sideways or slows it down.
- Continue forward once clear.

Obstacle Avoidance

- The green triangle = the agent.
- The orange and blue lines = feelers (direction sensors).
- The circles (A, B, C) = obstacles.
- The red arrow = the steering force to avoid collision.



Separation Behavior

What is it:

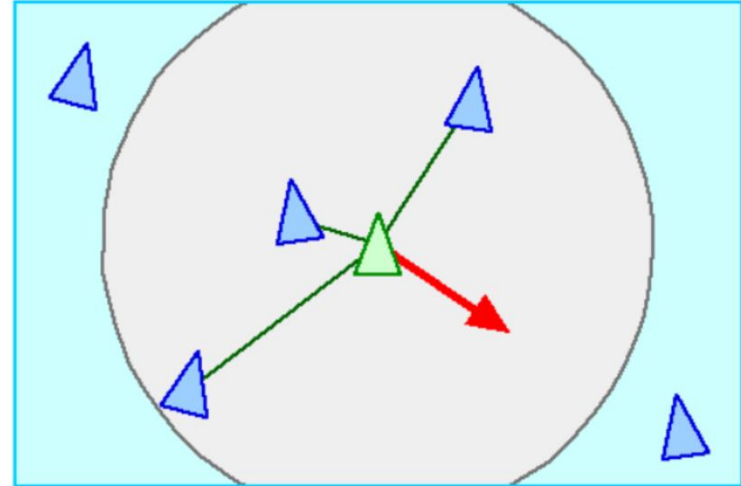
- Keep a comfortable distance from nearby agents, avoid crowding or overlapping.

Goal/Use:

- Used in flocks, crowds, or swarms to give each character personal space.
- **How It Works:**
 - For each nearby neighbor, calculate a small force that pushes the agent away.
 - Add all these forces together to get the total separation force.
 - Stronger push = closer neighbor.

Separation Behavior

- The gray circle = detection area around the agent.
- Blue triangles = neighbors.
- Green arrows = small pushes away from each neighbor.
- Red arrow = the combined steering direction to move away.



Cohesion Behavior

What is it:

- Cohesion makes agents move toward the center of their group.

Goal/Use:

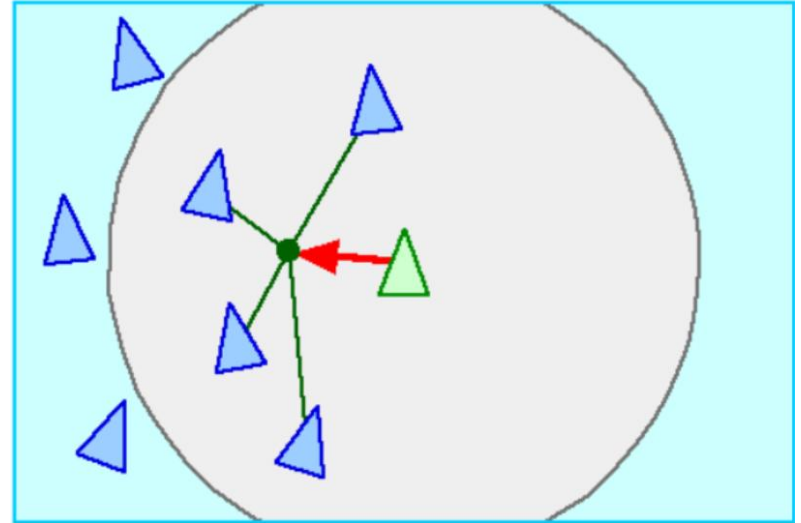
- Used to keep groups of characters together, like fish schooling or birds flocking..

- **How It Works:**

- Compute the average position of nearby agents.
- Seek toward that center point.

Cohesion Behavior

- The green dot = group's center.
- The green lines = connections to each neighbor.
- The red arrow = steering force toward the group center.
- The gray area = neighborhood range considered.



Alignment

What is it:

- Alignment makes agents face the same direction as their neighbors.

Goal/Use:

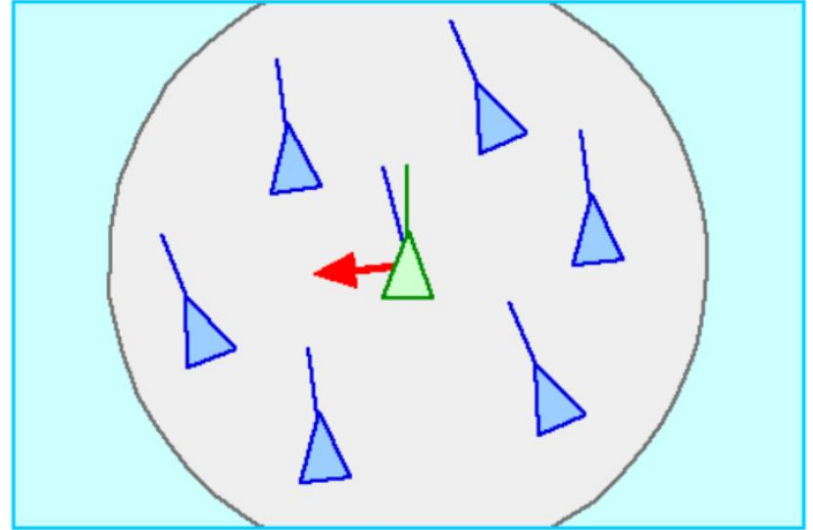
- Used so groups of agents travel smoothly in the same general direction (part of flocking).

- **How It Works:**

- Look at the velocity of nearby agents.
- Find the average direction they're moving.
- Steer to align your own direction to match theirs.

Alignment

- The blue triangles = neighbors with different headings.
- The red arrow = correction force to rotate toward the average heading.



Flocking / Leader Following

What is it:

- Flocking combines Separation + Cohesion + Alignment so agents move together as a natural, flowing group.
- Leader Following adds one more goal, stay behind and follow the leader.

Goal/Use:

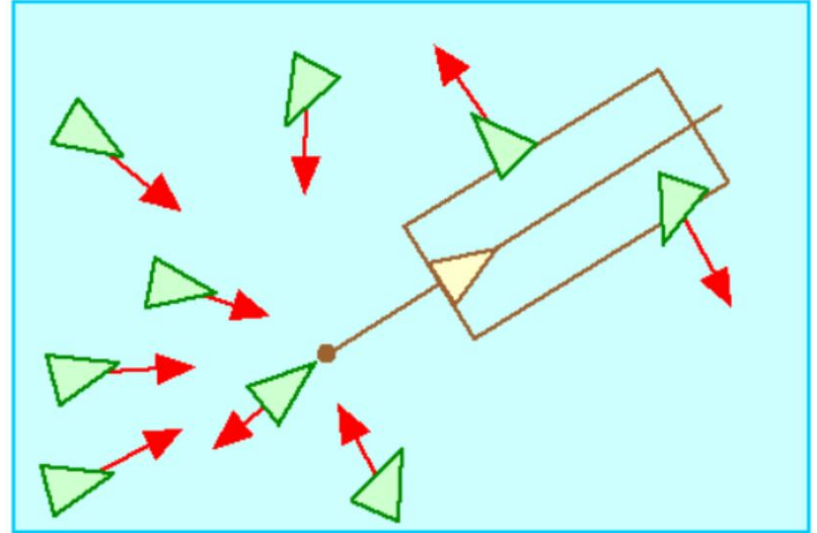
- Used for birds flying together, fish schooling, or followers moving behind a leader (like a squad).

- **How It Works:**

- Separation: don't bump into neighbors.
- Cohesion: stay with the group.
- Alignment: match the group's direction.
- Leader Following: seek a point behind the leader while avoiding it.

Flocking / Leader Following

- The green triangles = group members.
- The rectangle = the leader.
- The red arrows = steering forces combining all rules at once.



Flocking Behavior

Note that:

The flocking behavior relies on blending three simple steering behaviors:

- Separation: move away from characters that are too close
- Alignment: move in the same direction and at the same velocity as the group
- Cohesion: move toward the center of mass of the flock

In flocking behavior, separation is more important than cohesion, which is more important than alignment.

Combining Steering Behaviors

- Individually, steering behaviors can achieve a good degree of movement sophistication. However, by combining steering behaviors together, even more complex movement can be achieved.

There are two methods of combining steering behaviors: blending and arbitration.

- **Blending** combines behaviors by executing all the steering behaviors and combining their results using a set of weights.
- **Arbitration** selects one or more steering behaviors to have complete control over the character.





End of the Lecture – Thank you!

See you in class for tutorial.